

INFORME 1º: PROGRAMACIÓN DINÁMICA



PROFESOR:

M.Sc. Luis Antonio Selis Vasquez

SECCIÓN: Q

INTEGRANTES :

Guadalupe Arango Calet Dreik Código:20261098I

Herrera Moore Kiara Yadhira Código: 20263048I

Velazques Perez Yvan Eduardo Código: 2026305F

1. Concepto de Programación dinámica.

1.1. Definición

La programación dinámica es un método de solución de problemas que permiten descomponer un modelo matemático de gran magnitud, que puede ser muy difícil de resolver, en diversos problemas más pequeños que por lo general son de resolución mucho más fáciles. Además, el método de programación dinámica permite descomponer el problema de manera que una vez que se han resuelto los problemas menores se tiene la solución óptima para el problema mayor, principio de optimalidad. Cada uno de tales problemas más pequeños que se crean, se identifican con una etapa del procedimiento de solución de la programación dinámica. Con frecuencia, tales etapas se crean por el hecho de que se debe tomar una secuencia de decisiones en el transcurso del tiempo. En la mayor parte de los casos, no es posible considerar a estos problemas más pequeños como si fueran completamente independientes de los demás y es aquí en donde el método de la programación dinámica resulta útil.

1.2. Características

- **Subproblemas superpuestos.** El problema original puede dividirse en subproblemas que se repiten varias veces.
- **Subestructura óptima.** La solución óptima del problema global se construye a partir de soluciones óptimas de subproblemas (principio de optimalidad de Bellman).
- **Almacenamiento de resultados.** Guarda los resultados ya calculados para no repetir operaciones, mediante memoización (enfoque top-down, con recursión) o tabulación (enfoque bottom-up, con iteración y tablas).
- **Reducción del tiempo de ejecución.** Al evitar cálculos redundantes, puede transformar algoritmos de complejidad exponencial en polinomial.
- **Uso de memoria adicional.** A cambio de mejorar la velocidad, requiere más memoria para almacenar resultados parciales. Existe un equilibrio entre ambos recursos: a más memoria, menos tiempo de cálculo.

2. Principios de funcionamiento de la programación dinámica:

2.1. Subproblemas superpuestos.

Los subproblemas superpuestos implican que un algoritmo recursivo resuelve los mismos subproblemas más pequeños varias veces. Esta es una de las dos propiedades (junto con una subestructura óptima) que hacen que un problema sea idóneo para la programación dinámica, donde se almacenan en caché los resultados de los subproblemas para evitar recálculos.

Los problemas de programación dinámica a menudo presentan subproblemas superpuestos, donde la solución de un subproblema es utilizada por múltiples

subproblemas de mayor tamaño. La programación dinámica aprovecha esta repetición almacenando los resultados de los subproblemas y reutilizándolos cuando es necesario, lo que reduce los cálculos redundantes y mejora la eficiencia.

Ejemplo: Un caso típico es el cálculo de la sucesión de Fibonacci mediante recursión simple. Para obtener $\text{fib}(5)$ se calculan $\text{fib}(4)$ y $\text{fib}(3)$, pero $\text{fib}(4)$ a su vez vuelve a calcular $\text{fib}(3)$, y así sucesivamente. Como resultado, un mismo subproblema se resuelve múltiples veces y el número de llamadas crece de forma exponencial, con una complejidad de $O(2^n)$.

```
#include <stdio.h>

int llamadas = 0;

int fib(int n) {
    llamadas++;
    if (n == 0 || n == 1) {
        return n;
    }
    return fib(n - 1) + fib(n - 2);
}

int main(void) {
    int n;
    printf("Ingrese un numero: ");
    scanf("%d", &n);
    printf("Fibonacci de %d es: %d\n", n,
        fib(n));
    printf("Llamadas realizadas: %d\n",
        llamadas);
    return 0;
}
```

Código 1. Cálculo de Fibonacci con recursión simple y contador de llamadas.

Llamáramos a $\text{fib}(n-1)$ y $\text{fib}(n-2)$ subproblemas de $\text{fib}(n)$. Tomaremos como ejemplo el $n=5$, cuyo árbol de llamadas se muestra a continuación:

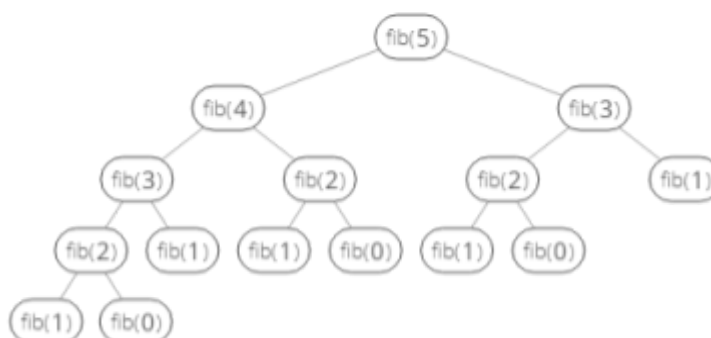


Figura 1. Árbol de llamadas de $\text{fib}(5)$.

Al ejecutar el programa con $n = 5$ se obtiene la siguiente salida:

```
Ingrese un numero: 5
Fibonacci de 5 es: 5
Llamadas realizadas: 15

=====
Process exited after 5.313 seconds with return value 0
Presione una tecla para continuar . . . |
```

Salida 1. Ejecución del programa con $n = 5$.

2.1.1. Complejidad del código

A continuación se analiza la complejidad temporal y espacial del programa del Código 1, que resuelve $\text{fib}(n)$ con recursión simple.

Complejidad temporal

El tiempo de ejecución de $\text{fib}(n)$ se expresa con la siguiente relación de recurrencia:

$$T(n) = T(n - 1) + T(n - 2) + 1, \text{ con } T(0) = T(1) = 1$$

Como $T(n - 2) \leq T(n - 1)$, la recurrencia se acota superiormente y se expande paso a paso:

$$T(n) \leq 2 \cdot T(n - 1) + O(1); \quad T(1) = O(1)$$

$$T(n - 1) \leq 2 \cdot T(n - 2) + O(1) \rightarrow T(n) \leq 4 \cdot T(n - 2) + 3 \cdot O(1)$$

$$T(n - 2) \leq 2 \cdot T(n - 3) + O(1) \rightarrow T(n) \leq 8 \cdot T(n - 3) + 7 \cdot O(1)$$

$$T(n) \leq 2^k \cdot T(n - k) + (2^k - 1) \cdot O(1)$$

$$\text{Con } k = n - 1: T(n) \leq 2^{n-1} \cdot T(1) + (2^{n-1} - 1) \cdot O(1)$$

$$T(n) \leq 2^{n-1} \cdot O(1) + (2^{n-1} - 1) \cdot O(1) = (2^n - 1) \cdot O(1) = O(2^n)$$

Por lo tanto, el tiempo de ejecución es $O(2^n)$: cada llamada realiza un trabajo constante, $O(1)$, y el número total de llamadas crece de forma exponencial. Para $n = 5$ esto equivale a las 15 llamadas observadas en el árbol.

Complejidad espacial

La recursión no almacena todos los nodos del árbol a la vez, sino solo las llamadas activas, es decir, las de una rama que va desde la raíz hasta el nodo que se está calculando. La rama más larga se obtiene siguiendo siempre $\text{fib}(n-1)$. Para $n = 5$:

$$\text{fib}(5) \rightarrow \text{fib}(4) \rightarrow \text{fib}(3) \rightarrow \text{fib}(2) \rightarrow \text{fib}(1)$$

En esta rama hay 5 llamadas activas simultáneamente y, en general, n . Como cada llamada ocupa un espacio constante en la pila (parámetro n , dirección de retorno y variables locales), el espacio total es $n \cdot O(1) = O(n)$. La variable global llamadas y las variables de main ocupan $O(1)$, por lo que la complejidad espacial del programa es $O(n)$.

2.2. Subestructura óptima

La subestructura óptima es una propiedad de ciertos problemas según la cual una solución óptima del problema global contiene, dentro de ella, soluciones óptimas de sus subproblemas (principio de optimalidad de Bellman, 1957). Gracias a ello, el problema puede resolverse planteando una recurrencia que combine los resultados óptimos de subproblemas más pequeños (Cormen et al., 2022). Esta propiedad, junto con la existencia de subproblemas superpuestos, hace que un problema sea adecuado para la programación dinámica: la subestructura óptima garantiza que la solución construida es correcta, y la superposición es la que permite ahorrar tiempo al almacenar y reutilizar resultados.

Ejemplo: Supongamos un mapa donde los tramos tienen estos costos: $A-B = 2$, $B-E = 3$, $A-C = 3$, $C-E = 4$ y $E-L = 4$ (Figura 2). La ruta más barata de A a L es $A \rightarrow B \rightarrow E \rightarrow L$, con costo $2 + 3 + 4 = 9$. Esta ruta pasa por E, y su tramo de A a E ($A \rightarrow B \rightarrow E$, costo 5) es también la ruta más barata de A a E, porque la alternativa $A \rightarrow C \rightarrow E$ cuesta 7. Si se reemplazara ese tramo por la ruta más cara, el total subiría a 11, lo que contradice que la ruta original fuera óptima. Es decir, la solución óptima de A a L contiene la solución óptima de A a E, que es un subproblema.

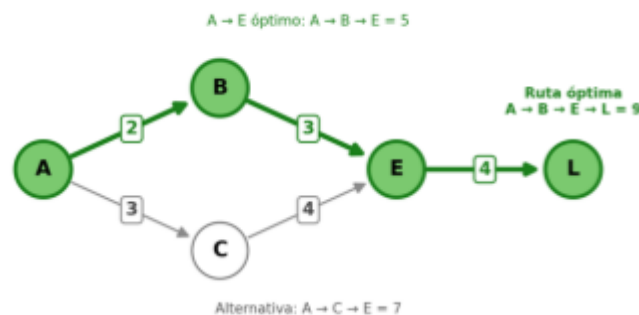


Figura 2. Mapa de rutas de A a L; en verde, la ruta de menor costo.

Se implementó un programa en C (adjunto en los entregables del informe como [subestructura_optima.cpp](#)) que permite ingresar interactivamente los costos de los cinco tramos y verificar experimentalmente la propiedad. El algoritmo evalúa las rutas posibles de A a E, selecciona la óptima local y demuestra cómo la elección del subcamino más costoso degrada la solución global hacia L. Su ejecución se presenta en la Salida 2:

```
Costo del tramo A-B: 2
Costo del tramo B-E: 3
Costo del tramo A-C: 3
Costo del tramo C-E: 4
Costo del tramo E-L: 4

Tramo A -> E por B: 5
Tramo A -> E por C: 7
Mejor tramo A -> E: 5

Ruta optima de A a L: A -> B -> E -> L
Costo minimo: 9

Con el tramo mas caro el total seria: 11 (peor)

-----
Process exited after 70.27 seconds with return value 0
Presione una tecla para continuar . . . |
```

Salida 2. Ejecución del programa.

Esta propiedad no la cumple cualquier problema: por ejemplo, la ruta simple más larga entre dos puntos de un mapa no la tiene. Más adelante se verá que el camino mínimo en una cuadrícula sí la cumple y, además, presenta subproblemas superpuestos, por lo que se resuelve con programación dinámica.

2.2.1. Complejidad del Código

A continuación se analiza la complejidad temporal y espacial del programa del Código 2.

Complejidad temporal

El programa lee cinco costos, realiza dos sumas, dos comparaciones y muestra unos pocos resultados. Son operaciones simples, y su cantidad es fija: no depende de los valores ingresados ni crece con ningún tamaño de entrada. Por lo tanto, el tiempo de ejecución es una constante, $T = c \cdot O(1) = O(1)$.

Complejidad espacial

Solo se almacenan nueve variables enteras (AB, BE, AC, CE, EL, por_B, por_C, mejor_AE y peor_AE), una cantidad fija. Por lo tanto, la complejidad espacial es $O(1)$.

Este resultado es esperable, pues el ejemplo ilustra la subestructura óptima en un mapa reducido y directo. Cabe destacar que, a simple vista, para el ojo humano resulta evidente deducir cuál es la ruta de menor costo en un grafo de pocos nodos; sin embargo, el computador no posee esta intuición visual y debe procesar de manera secuencial los tramos, acumular los costos y realizar formalmente cada comparación lógica. Si el problema escala y se agregan múltiples nodos y bifurcaciones intermedias —como ocurrirá más adelante en problemas como el camino mínimo en una cuadrícula—, la inspección humana se vuelve imposible. Bajo un enfoque de fuerza bruta que explore exhaustivamente todas las rutas posibles, la complejidad temporal colapsaría computacionalmente al crecer de forma exponencial $O(k^V)$, donde k representa el factor de ramificación medio y V el número de vértices o decisiones intermedias. Es allí donde cobra real trascendencia la subestructura óptima: al permitir que la programación dinámica evite la exploración redundante de caminos completos y construya la solución de manera iterativa apoyándose en las soluciones óptimas de los subproblemas previamente calculados, la complejidad temporal se reduce dramáticamente a un rango polinomial u optimizado de $O(V + E)$ o $O(V^2)$, haciendo factible la resolución eficiente del problema a gran escala.

3. Modelado y métodos de implementación.

3.1. Memoización:

La memoización (popularmente llamado memorización) es una técnica de optimización de algoritmos basado en el almacenamiento de cálculos recursivos o repetitivos, permitiendo reducir tiempos de cálculo al no repetir los mismos subproblemas si estos ya han sido anteriormente calculados. De esta manera, el programa reutiliza constantemente los cálculos previamente guardados en la memoria. Siendo muy eficiente para conjuntos de entrada relativamente reducidos.

De forma práctica y retornando al caso del cálculo de la sucesión de Fibonacci para $\text{fib}(5)$, el proceso de resolución mediante memoización se realizaría de la siguiente forma:

- Para calcular $\text{fib}(5)$, busca en la memoria si ya está guardada.
- Al no estar guardado, calcula $\text{fib}(5) = \text{fib}(4) + \text{fib}(3)$.
- En orden, repite el proceso para la rama del $\text{fib}(4)$, guardando en memoria cada cálculo: $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$.
- Una vez finalizado, calcula la rama $\text{fib}(3)$ aplicando el mismo proceso, con la diferencia que esta vez si está guardado.
- Reemplaza automáticamente el valor de $\text{fib}(3)$.

De esa forma los valores guardados en la memoria, en orden, serían los siguientes $\{ \text{fib}(5), \text{fib}(4), \text{fib}(3), \text{fib}(2), \text{fib}(1), \text{fib}(0) \}$. El método permite eliminar todos los llamados correspondientes a la rama $\text{fib}(3)$, pero también los correspondientes a las ramas análogas $\text{fib}(2)$ y $\text{fib}(1)$ que se visualizan en el árbol de llamadas presentado a continuación.

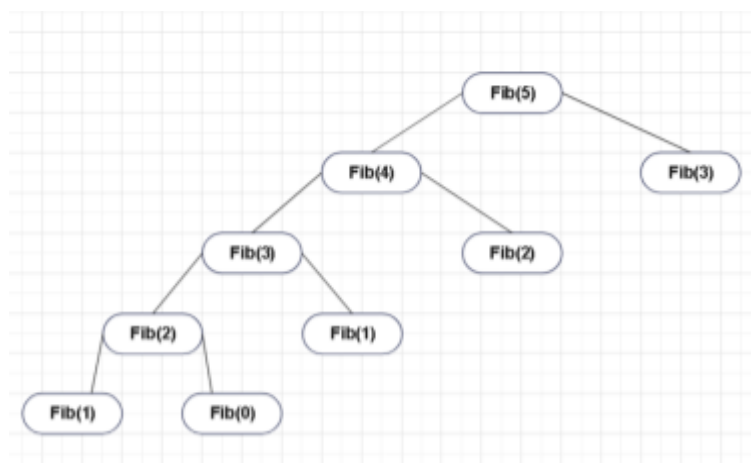


Figura 3. Árbol de llamadas de $\text{fib}(5)$ usando memoización.

En lo que respecta a complejidad temporal, tal y como se puede apreciar, el tiempo de ejecución de $\text{fib}(n)$ pasa de ser $O(2^n)$ a $O(n)$ al igual que la complejidad espacial. Ya que en su ejecución, se calcula primero cada valor desde n hasta 1,

dejando el resto de llamadas como instantáneas y recuperando el valor correspondiente en un tiempo $O(1)$. El total de llamadas que realiza el programa es de $2n - 1$ (n llamadas de la rama principal y $n-1$ de las llamadas recursivas).

```
int memo(int n, int cache[]) {
    llamadas++;

    if (n <= 1) {
        return n;
    }

    if (cache[n] != -1) {
        return cache[n];
    }

    cache[n] = memo(n - 1, cache) + memo(n - 2, cache);

    return cache[n];
}

int fib(int n) {
    int cache[n+1];

    for (int i = 0; i <= n; i++) {
        cache[i] = -1;
    }

    return memo(n, cache);
}
```

Código 2. Función para el cálculo de Fibonacci con memoización

En este caso, si bien el número de llamadas deja de crecer de forma exponencial para pasar a crecer linealmente, la aplicación de esta técnica implica el sacrificio del espacio en memoria. A pesar de que ambos métodos tienen la misma complejidad espacial, el proceso sin memoización utiliza un espacio de $O(n)$ únicamente en la profundidad máxima de la pila de llamadas, mientras que con memoización ese espacio se mantiene constante al utilizarlo para guardar todos los valores (desde $n-1$ hasta 0) además de la necesaria para la pila de llamadas.

Aún así, el sacrificio de memoria es insignificante en comparación con la reducción de la complejidad temporal. Para el caso simple de $\text{fib}(5)$ no es muy significativo, debido a que el total de llamadas se reduce de 15 a 9. Sin embargo, para números mayores como $n=20$ en lugar de realizar más de 1 millón de llamadas ($2^{20}-1$) solo realiza 39 ($2 \cdot 20 - 1$).

```
Ingrese un numero: 5
Fibonacci de 5 es 5
Numero de llamadas a la funcion fib: 9
Ingrese un numero: 20
Fibonacci de 20 es 6765
Numero de llamadas a la funcion fib: 39
```

Salida 3. Ejecución del código 2 para $n=5$ y $n=20$

El principal problema de la memoización aparece cuando el programa a ejecutar exige una escalabilidad masiva en profundidad. Esto, ya que al utilizar valores demasiado grandes incrementa el consumo de memoria necesario para guardar cada valor necesario a pesar de que el proceso sea sencillo. Por lo que la memoización funciona de manera óptima en los problemas para los que existen combinaciones imposibles o que tienen un orden topológico estricto (estructura compleja o irregular) como pueden ser los de teoría de juegos o búsqueda de rutas, siempre y cuando se tenga una profundidad controlada y el sistema pueda soportar la carga de memoria.

3.2. Tabulación:

Es una técnica que busca principalmente la resolución del problema partiendo de su orden topológico y resolviendo cada subproblema de forma iterativa usando los casos iniciales hasta conseguir el resultado pedido. A diferencia de la memoización, que no depende del orden topológico del problema, para la tabulación si es necesario desglosar dichos subproblemas para que sea posible calcularlo desde lo más elemental (orden inverso). Este método reduce el tiempo de cálculo al solo guardar los datos necesarios en una "tabla" para continuar la secuencia utilizando bucles *for* o *while*, optimizando a su vez la complejidad espacial empleada por el problema. Por lo que el método deja de depender de la recursividad y mejora más eficientemente la complejidad y escalabilidad de los problemas.

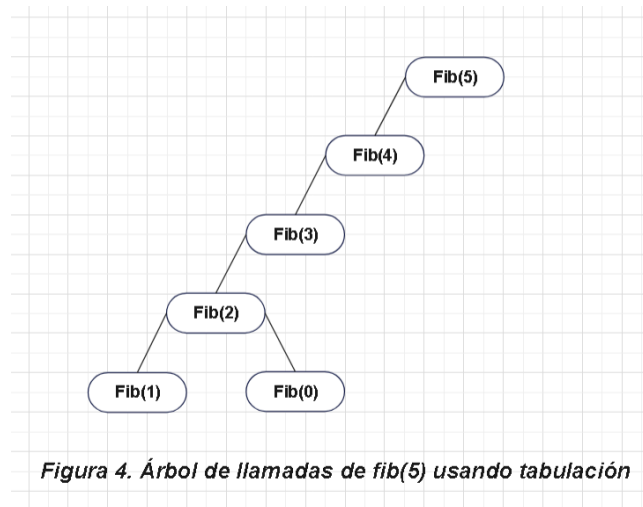
Esta forma de almacenar los datos también se le conoce como "la ventana deslizante" (sliding window) ya que va desplazando los datos conforme va avanzando. De forma general, si para el caso $f(i)$, la función depende de k estados anteriores $f(i-a_1)$, $f(i-a_2)$, $f(i-a_3)$, ..., $f(i-a_k)$; entonces la tabla tiene tamaño k y cuando la función calcule el valor siguiente $f(i+b)$ se desplazan todos los valores calculados guardando los nuevos valores en los espacios correspondientes y eliminando los residuos o valores que ya no se usarán. Entonces el tamaño de la tabla es una constante cuyo valor es k .

En general, esta optimización permite, además de reducir la complejidad temporal (de $O(2^n)$ a $O(n)$, o en su defecto $O(n \cdot k)$), poder disminuir enormemente la complejidad espacial que requiere el problema. El tamaño de la memoria auxiliar usada pasa de n a k , pero, al ser k una constante, la complejidad espacial se reduciría a $O(1)$ consiguiendo que el programa funcione para números muy altos sin el riesgo de sobrepasar la memoria.

Para el caso de Fibonacci, como la función es $fib(n) = fib(n-1) + fib(n-2)$, el tamaño de la matriz o tabla es de 2, solo se guardan esos valores en variables temporales (ya sean *prev1* y *prev2*), o en un arreglo que va ir reemplazando los valores de la siguiente forma:

- Calcula los valores iniciales: $fib(0) = 0$ y $fib(1) = 1$ y los almacena, sean esos valores $a_0 = 0$ y $a_1 = 1$ (el flujo es de a_1 hacia a_0).

- Calcula el valor siguiente: $\text{fib}(2) = \text{fib}(1) + \text{fib}(0) = 1$. Y desplaza los resultados: $a_0 = 1$ y $a_1 = 1$.
- Continúa con el siguiente $\text{fib}(3) = \text{fib}(2) + \text{fib}(1) = 2$ y $a_0 = 1$ y $a_1 = 2$.
- Así sucesivamente hasta llegar a $\text{fib}(n)$.



Durante el proceso solo se usa de manera constante el espacio para almacenar $\{a_0; a_1\}$. De igual forma la complejidad temporal queda reducida a $O(n)$ al realizarse tantas llamadas como el número pedido, en específico, se realizan $n-1$ llamadas ya que $\text{fib}(0)$ y $\text{fib}(1)$ al ser las bases son instantáneas, y para los siguientes números (del 2 al n) son $n-1$ llamadas. Si no usáramos la ventana deslizante, la complejidad espacial sería de $O(n)$ al guardar cada dato calculado. Sin embargo, con el uso del mismo, la complejidad espacial pasaría a ser de $O(1)$ al ser un arreglo fijo y de pocas variables fijas.

```

#include <stdio.h>

int llamadas = 0;

int fib(int n) {
    int cache[3];
    cache[0] = 0; // fib(n-2)
    cache[1] = 1; // fib(n-1)
    cache[2] = 0; // fib(n)

    for (int i = 2; i <= n; i++) {
        llamadas++;

        cache[2] = cache[1] + cache[0];
        cache[0] = cache[1];
        cache[1] = cache[2];
    }

    return cache[2];
}

```

Código 3. Función para el cálculo de Fibonacci con tabulación.

El espacio para este ejemplo es de 3 variables debido al uso de la ventana deslizante, debido a la necesidad de guardar el nuevo valor $\text{fib}(n)$ para poder desplazarse posteriormente a las variables correspondientes de manera práctica. Aun así, se pueden usar solo 2 variables realizando el cambio dentro del bucle *for* reemplazando las 3 últimas líneas por las 2 siguientes:

- $\text{cache}[1] = \text{cache}[1] + \text{cache}[0]$
- $\text{cache}[0] = \text{cache}[1] - \text{cache}[0]$

```
Ingrese un numero: 5
Fibonacci de 5 es 5
Numero de llamadas a la funcion fib: 4
Ingrese un numero: 20
Fibonacci de 20 es 6765
Numero de llamadas a la funcion fib: 19
```

Salida 3. Ejecución del código 3 para $n=5$ y $n=20$

La principal ventaja de este método es el prescindir por completo de la recursividad mejorando en gran medida el rendimiento del hardware (bajo consumo de memoria), lo que lo vuelve el método más óptimo para problemas que requieran calcular gran cantidad de subproblemas o cuando se conoce el orden topológico del mismo. Aún así, frente a la memoización, se vuelve ineficiente para problemas con restricciones o combinaciones en gran cantidad al calcularse valores que no corresponden al proceso de la función, o para problemas con gran complejidad o dimensionalidades grandes al no poderse mapear con claridad las bases o que la tabla colapse la memoria debido a la masividad del espacio que engloba el problema.

3.3. Estados y transiciones:

Lorem ipsum dolor sit amen, dispisencis consequetur amet in.

3.3.1. Definición de “Estado” (S):

En programación dinámica, los estados representan la información mínima necesaria con la que se realiza una decisión o llamada en el paso i . Solo se define en base a lo suficiente para poder determinar los pasos siguientes. Es la unidad necesaria para describir un subproblema durante el proceso de resolución. Dentro de su definición cumple con las propiedades de la subestructura óptima y la determinación de la dimensionalidad del problema.

Notación formal del Estado

Comúnmente se representan por “s” para un estado individual y “S” para el espacio de los estados globales que ocupa todos los posibles subproblemas necesarios para la resolución del problema.

$s = \{v_1, v_2, v_3, \dots, v_k\}$. Siendo k la dimensionalidad.

El significado que se le dé al Estado influye directamente en el diseño de la solución llegando a ser la parte más complicada del proceso. Esto debido a que si contiene información innecesaria o se encuentra incompleto, la solución planteada deja de funcionar o directamente deja de ser aplicable

Aplicación de los Estados

De manera simplificada, para el caso de la sucesión de Fibonacci, el estado vendría a ser $fib(i)$ para i -ésimo número de Fibonacci. No es necesario determinar el recorrido del programa hasta llegar a dicho valor, el estado solamente engloba el subproblema:

$$fib(i) = fib(i-1) + fib(i-2)$$

Al ser de una variable, el problema es unidimensional y esa única variable establece en qué parte del proceso te encuentras siendo $s = \{n\}$. Por lo que para calcular $fib(n)$ solo necesitas saber el índice n , lo que permite reducir el tamaño del conjunto de estados a solo 2 o 3 tal y como se observa en la sección de tabulación (3.2).

Para este caso un error por parte de la definición del estado sería incluir el arrastre de los valores debido a la aplicación de la memorización, en esencia, haciendo:

$$s = \{n, [fib(0), fib(1), fib(2), \dots, fib(n-1)]\}$$

Ya que el programa empieza a almacenar datos innecesarios cuando solo se requieren los 2 últimos valores. De tal forma que se incremente de manera constante el espacio utilizado en memoria y añadiendo complejidad innecesaria al problema.

Otro caso de error al momento de definir los estados sería la inclusión de parámetros innecesarios ya sean pasos totales o el número de procesos de recursividad calculados, entre muchos otros. Sea el caso:

$$s = \{n, p, r\}. \text{ Siendo } p \text{ los pasos totales y } r \text{ las llamadas recursivas.}$$

tanto p como r no aportan cambios reales dentro del problema, siendo que lo único generado son combinaciones inexistentes (por ejemplo, el sistema ve como diferentes pero posibles a $fib(5,30,0)$ que a $fib(5,9,3)$ a pesar de que la primera es absurda) y que la programación dinámica y la optimización pierdan propósito dentro de la solución.

A modo de síntesis, es vital la correcta parametrización y definición de los estados, puesto que permite establecer con certeza la estructura u orden

topológico y el método más adecuado para su resolución. Optimizando correctamente la dimensionalidad, los subprocesos y la complejidad del problema

3.3.2. Definición de “Transición” ($\rightarrow$):

Una vez que se tiene correctamente definido y delimitado los estados(S) del problema. La siguiente parte vital del diseño o del modelado de la solución se encuentra al delimitar las transiciones, las cuales permiten conectar los estados entre sí .

Definición y notación

Como definición, una transición es una regla lógica que formaliza la dependencia entre estados. Sea el espacio de estados totales S , si un estado $s \in S$ se debe calcular a partir de un conjunto (subconjunto) de estados anteriores sea s' , la transición se denota como:

$$s' \rightarrow s$$

Se puede expresar el conjunto de transiciones o dependencias como un grafo dirigido acíclico (DAG) en donde los nodos representan cada estado individual y las aristas cada una de las transiciones entre ellos. Estructura en donde cada estado conecta con otro de forma unidireccional (dirigido), y no existe ruta de transiciones que retorne a un valor ya calculado (acíclico, no existen combinaciones $(s \rightarrow s' \rightarrow s)$) y permite organizar cada elemento mediante jerarquía u orden topológico de tal forma que se pueda evaluar los casos base e ir avanzando de forma precisa sin la presencia de bucles infinitos.

Aplicación de Transiciones y Recurrencias

La ecuación de recurrencia es la expresión algebraica que ayuda a observar las transiciones de manera en que son computable. Para el caso de la sucesión de Fibonacci es el siguiente:

$$F(n) = F(n-1) + F(n-2); \forall n \geq 2$$

En el análisis de las transiciones para resolver el estado $s = (n)$ solo se requiere de consultar estrictamente los 2 estados anteriores $s_1 = (n-1)$ y $s_2 = (n-2)$.

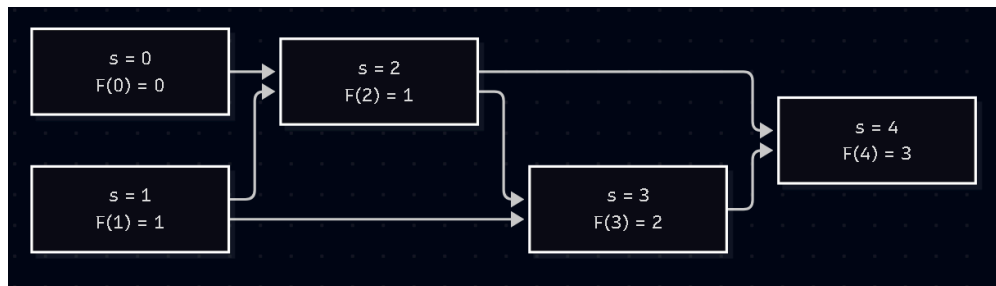


Figura 5: DAG de dependencias para el cálculo de Fibonacci para el caso $n = 5$.

Para casos de 2 o más dimensiones se utiliza el mismo análisis, con la diferencia de que el planteamiento de las ecuaciones de recurrencia son más complejos a

medida que se incrementa la dimensionalidad. Con respecto a los casos de problemas bidimensionales, un ejemplo de ello es el problema del cambio de moneda. Problema en el que se pide dar un cambio de un monto V usando la menor cantidad de monedas de denominaciones $C = \{c_1, c_2, \dots, c_k\}$. Cuya ecuación de recursividad es:

$$DP(v) = \min_{c_i \leq v} (DP(v - c_i) + 1)$$

Dicha ecuación lo que hace es calcular el el número de monedas a partir del número de monedas de ese mismo monto restando una moneda de las brindadas ($c_i \leq v$ y $DP(v - c_i)$), por ello se le suma uno para contar esa moneda restada. Sin embargo, al existir varias posibilidades, se debe probar con cada moneda y después comparar las cantidades para seleccionar el mínimo de ellos ($\min_{c_i \leq v}$).

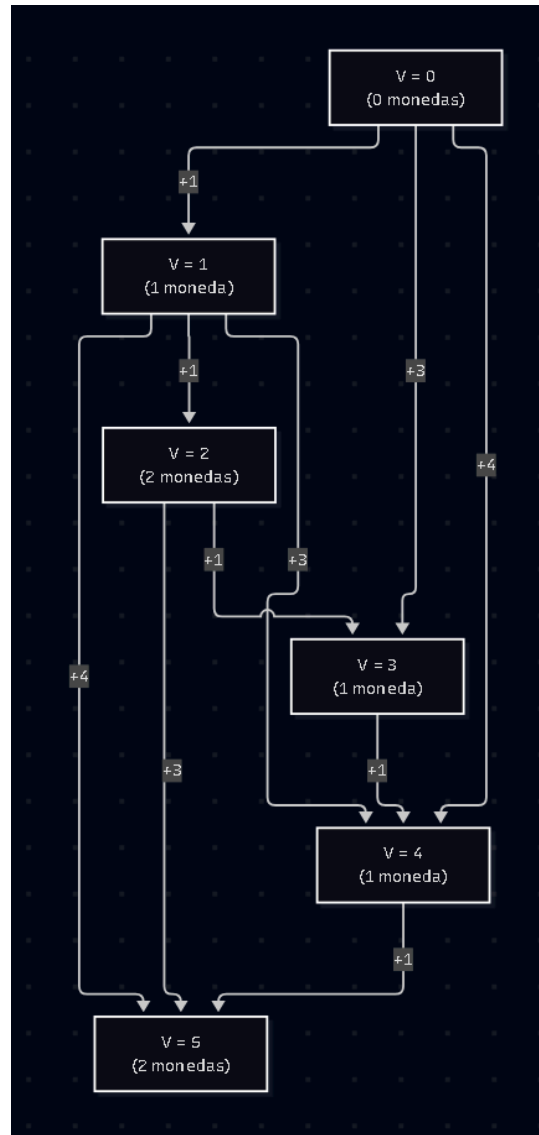


Figura 5: DAG de dependencias para el cálculo del cambio de moneda para el caso $V = 5$ y $C = \{1, 3, 4\}$.

Queda demostrada la importancia de delimitar correctamente las transiciones, para el caso de la sucesión de Fibonacci, permitía reducir el espacio de estados totales a una pequeña cantidad fija. Y para el caso del problema del cambio de moneda, permite reducir el problema a una sola variable (v) gracias a que el programa recorre todas las opciones de las denominaciones (C). De esa manera, su aplicación adecuada posibilita asegurar que el camino óptimo sea siempre el elegido por parte de la solución.

4. Enfoque Top-Down

Consiste en definir el problema original y, de allí, crear algoritmos que de forma recursiva puedan dividir el problema y segmentarlo en situaciones más pequeñas y fáciles de manejar. Una vez que se hayan creado estos subproblemas, es necesario almacenarlos; para ello, el programador se puede valer de diferentes herramientas como un arreglo, una matriz o una tabla hash. Este almacenamiento, será de mucha ayuda porque la idea es que estos subproblemas aparezcan nuevamente con el avance del algoritmo, y cuando lo hagan, se pueda utilizar la memoización para llamar a su valor almacenado y así completar la operación.

Este enfoque puede reducir la complejidad computacional de varios problemas y mejorar la eficiencia de los algoritmos respectivos. Se recomienda usar cuando se pueden encontrar fácilmente relaciones de recurrencia; además, se debe considerar la existencia de subproblemas que se repitan continuamente y que cumplan con la propiedad de la subestructura óptima. Por otro lado, suele ser más fácil de implementar que el enfoque bottom-up debido a que es más intuitivo y se trabaja desde el objetivo principal.

Aplicaciones reales de este enfoque son la construcción de código para acciones de personajes de videojuegos, la construcción de mapas y algunos problemas de inversión en la bolsa de valores. La complejidad depende del problema que se esté analizando.

5. Enfoque Bottom-Up

Es el enfoque inverso al de Top-Down, porque se empieza con los casos base y se trata de llegar al problema original y de construir así la solución. Además, este enfoque no utiliza la recursividad, sino la iteración, por lo que presenta una tabla de PD (Programación Dinámica). Asimismo, este método resulta particularmente útil en los casos donde se tiene que calcular todos los estados/subproblemas del proceso dinámico, y cuando se observa que la memoización no se ajusta al problema y genera complejidad innecesaria con muchas llamadas recursivas (lo cual puede causar desbordamiento de pila).

Entre algunas aplicaciones reales de este enfoque, están la resolución de comparación de cadenas, correctores ortográficos y reconocimiento de texto. La complejidad depende del problema que se esté analizando.

Para comprender mejor este enfoque y también el Top-Down, se usará el ejemplo de la sucesión de Fibonacci:

Fibonacci, Top-Down con memoización

Se puede afirmar que si “n” es N° de término:

$$F(0) = 0; F(1) = 1; F(n) = F(n-1) + F(n-2)$$

De lo anterior, se puede observar que se descomponen los términos más grandes, y se guardan los resultados en la memoria caché. Por ello, cuando se pide un término en cierta posición, solamente se recurre a la memoria almacenada de los dos términos anteriores y se suman. Para este problema, su complejidad computacional es $O(n)$; la demostración es la siguiente:

Complejidad computacional = Número de estados x trabajo por estado

$$T(n) = (n+1) \times O(1)$$

Como la memorización hace posible almacenar los resultados, cuando se quiera el valor de un término de posición “n”, se tendrán que hacer “n” búsquedas respectivas. Por lo tanto, la complejidad es lineal.

$$T(n) = O(n).$$

Fibonacci, Bottom-up

Es al revés:

$$F(0) = 0; F(1) = 1; F(n)+F(n+1) = F(n+2)$$

$$F(1) + F(2) = F(3).$$

Se utilizan los números más pequeños para armar los términos mayores. De esta manera se crea una lista donde se almacenan los valores respectivos. También, de igual manera que en el enfoque Top-Down, la complejidad computacional es $O(n)$; la demostración es la siguiente:

$$T(n) = n \cdot O(1)$$

$$T(n) = O(n)$$

Esto es porque en la serie de Fibonacci el algoritmo ya tiene el valor de $F(1)$ y de $F(2)$, por lo que a partir de allí calcula el valor de los demás valores a partir de la suma y lo guarda en una matriz para recordarlo consecutivamente, de esa forma hace un número de “n-2” operaciones que equivale a una complejidad $O(n)$.

6. Problemas clásicos

6.1 Camino mínimo en una cuadrícula cuadrada

Este problema trata de encontrar el mejor camino en una cuadrícula desde la esquina superior izquierda hasta la esquina inferior derecha; sin embargo, cada cuadrícula tiene un costo diferente para poder pasar; además, solo se puede ir hacia la derecha y hacia abajo.

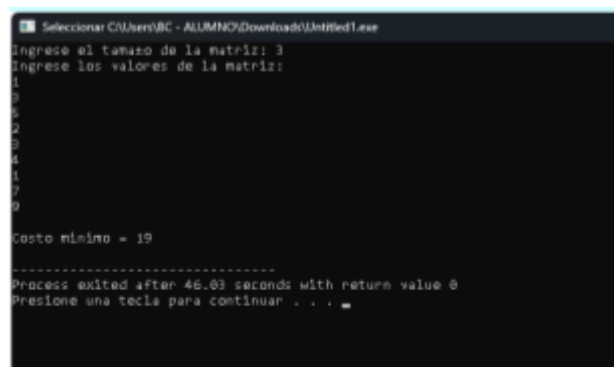
Para resolver este problema se utiliza un estado DP (Dynamic Programming) que representa al subproblema con el que se va a trabajar; en este caso, el subproblema es el costo mínimo para llegar al destino final. Se puede simbolizar de la siguiente manera:

$$DP[i][j] = C[i][j] + \min(DP[i-1][j], DP[i][j-1])$$

Donde i = Número de fila, j = Número de columna, $C[i][j]$ representa el costo de la última celda y $\min(DP[i-1][j], DP[i][j-1])$ sería ya la expresión de la recurrencia y simboliza el coste mínimo de las celdas anteriores.

Su complejidad computacional se calcula en base al número de celdas. En el caso de que haya $n \times n$ celdas, la complejidad sería $O(n^2)$.

Se muestra el ejemplo de una solución de este problema en el lenguaje C, plataforma Dev-C.



```
Selección C:\Users\BC - ALUMNO\Downloads\Untitled1.exe
Ingrese el tamaño de la matriz: 3
Ingrese los valores de la matriz:
1 3 1
1 5 1
4 2 1

Costo mínimo = 19

-----
Process exited after 46.03 seconds with return value 0
Presione una tecla para continuar . . .
```

Salida 4. Ejecución de camino mínimo en una cuadrícula cuadrada

6.2 Mochila 0/1

El problema consiste en que se tienen varios objetos de diferentes pesos y valores, y se los quiere poner en una mochila que soporta un peso máximo de manera que la suma de los valores sea máxima. El nombre 0/1 es una analogía al código binario, donde 0 significa no tomar el objeto y 1 tomarlo.

En este sentido, se analiza el estado DP de las dos posibilidades . Para el caso de “no tomar el objeto”, una representación sería:

$$DP[i][w] = DP[i-1][w]$$

De lo anterior, “DP[i][w]” es el máximo valor que se puede obtener si se utiliza una cantidad “i” de objetos cuyo peso total es “w”.

Para el caso de “tomar el objeto”, una representación sería:

$$DP[i][w] = v_i + DP[i-1][w-w_i]$$

Donde “v_i” es el valor del objeto que se ha tomado y “w_i” es el peso del objeto, respectivamente.

De esta manera, si unimos los dos casos, quedaría:

$$DP[i][w] = \max(DP[i-1][w], v_i + DP[i-1][w-w_i])$$

Aquí ya se tiene que comparar los estados DP de los 2 casos y encontrar el mayor, para ello se realiza un número constante de pasos, al cual le llamamos “c”.

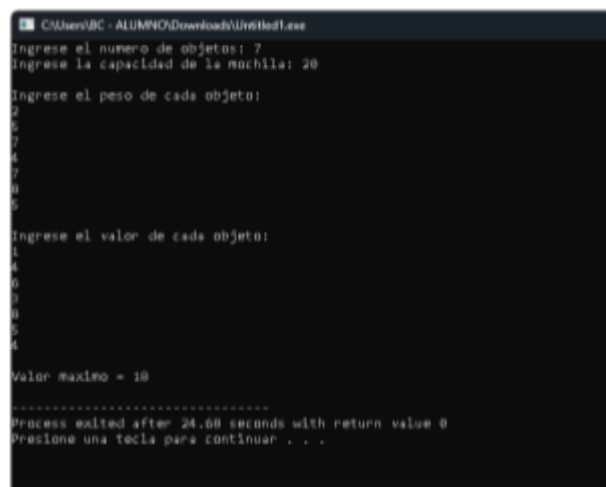
La complejidad computacional para resolver este problema con programación dinámica, se muestra de la siguiente manera:

$$T(N,W) = \sum_{i=1}^I \sum_{w=0}^W c = \sum_{i=1}^I (cW + 1) = cIW + I \quad \text{Como cada operación tiene un costo } O_{(1)}.$$

Entonces por propiedades de Big-O: $T(N,W) = O(IW)$.

Asimismo, para un mejor entendimiento, se aclara que “I” es igual al número de objetos y “W” es la capacidad total de la mochila. Finalmente, si el valor total y la cantidad de objetos se asemejan, se podría hablar de una complejidad $O(n^2)$.

Se muestra parte de una solución de este problema en el lenguaje C, plataforma Dev-C.



```
C:\Users\BC\ALUMNO\Downloads\Untitled1.exe
Ingrese el numero de objetos: 7
Ingrese la capacidad de la mochila: 20

Ingrese el peso de cada objeto:
2
5
7
4
7
8
9

Ingrese el valor de cada objeto:
1
4
0
3
0
6
8

Valor maximo = 18
.....
Process exited after 24.68 seconds with return value 0
Presione una tecla para continuar . . .
```

Salida 5. Ejecución de mochila 0/1

6.3 LCS (Longest Common Subsequence)

Este problema trata de hallar la longitud de la subsecuencia más larga en común entre 2 cadenas. Una subsecuencia es una parte de una cadena de texto; empero, sus elementos no necesariamente tienen que ser consecutivos en la cadena original. Un ejemplo sencillo es hallar la subsecuencia más larga que comparten las cadenas “Garantía” y “Gato”, la cual es “Gat” porque las letras “G”, “a” y “t” se repiten en ese orden en ambas cadenas, aunque no consecutivamente en el caso de “Garantía”.

Para llevar el problema a un estado DP, se define la cadena X de “m” elementos y la cadena Y de “n” elementos. Ahora, se analizan los 2 únicos casos.

Cuando los caracteres son iguales:

$$DP[i][j] = DP[i-1][j-1] + 1$$

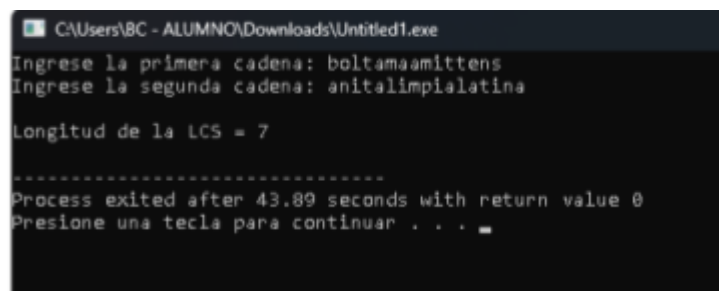
Donde “i” es la cantidad de los primeros elementos que se han contabilizado en la cadena X. El término “j” tiene la misma función, pero aplicado a la cadena Y. Además, $DP[i][j]$ es un subproblema que representa la subsecuencia más larga en común encontrada hasta los “i” caracteres de X y los “j” caracteres de Y.

Cuando los caracteres son diferentes:

$$DP[i][j] = \max(DP[i-1][j], DP[i][j-1])$$

La complejidad computacional se puede encontrar observando el número de elementos de cada cadena. Puesto que el número de operaciones es constante, se tiene que habrá $m \times n$ estados, y como cada estado cuesta $O(1)$, la complejidad total será de $O(m \times n)$.

Se muestra parte de una solución de este problema en el lenguaje C, plataforma Dev-C.



```
CAUsers\BC - ALUMNO\Downloads\Untitled1.exe
Ingrese la primera cadena: boltamaamittens
Ingrese la segunda cadena: anitalimpialatina

Longitud de la LCS = 7

.....
Process exited after 43.89 seconds with return value 0
Presione una tecla para continuar . . .
```

Salida 6. Ejecución de LCS (Longest Common Subsequence)

6.4 Distancia de edición

Para resolver este problema, se debe hallar la cantidad mínima de modificaciones que se deben realizar de manera que se transforme una cadena de texto en otra. Para aplicar la programación dinámica, como en el caso anterior, se conciben 2 cadenas X y Y de “m” y “n” elementos, respectivamente; también se construye el

estado $DP[i][j]$ que representa justamente el número de cambios mínimos para transformar los primeros “i” caracteres de X en los primeros “j” de Y. Si se encuentran elementos diferentes, se procede a realizar lo siguiente:

$$DP[i][j] = 1 + \min(DP[i-1][j], DP[i][j-1], DP[i-1][j-1])$$

De esta manera, cuando el algoritmo identifica 2 caracteres que son diferentes, usa el enfoque Top-Down y la recurrencia para desmembrar el problema hasta llegar al subproblema básico (en este caso sería $DP[1][1]$) y esto lo guarda en la memoria caché. El razonamiento consiste en que, si los caracteres son iguales, no hay costo computacional y se sigue con el trabajo; pero, si son diferentes, se tendrá que incrementar en 1 el número de cambios mínimos; a su vez, siempre se evalúan las transformaciones mínimas de los estados DP anteriores.

El coste computacional se obtiene del mismo razonamiento del caso anterior, y es $O(m \times n)$.

Se muestra parte de una solución de este problema en el lenguaje C, plataforma Dev-C.

```

C:\Users\BC - ALUMNO\Downloads\Untitled1.exe
Ingrese la primera cadena: AdiosJuanito
Ingrese la segunda cadena: odiospatito

Longitud de la LCS = 8

-----
Process exited after 75.78 seconds with return value 0
Presione una tecla para continuar . . .
  
```

Salida 7. Ejecución de Distancia de edición

6.5 Cambio de monedas

El problema es sobre el número mínimo de monedas, de ciertos valores, necesarias para alcanzar cierta cantidad de dinero. En este sentido, los valores de las monedas son : $\{c_1, c_2, c_3 \dots c_i\}$ y la cantidad que se quiere alcanzar es “X”. En estado DP sería de la siguiente forma:

$$DP[x] = \min_{c_i < x} (DP[X - c_i] + 1)$$

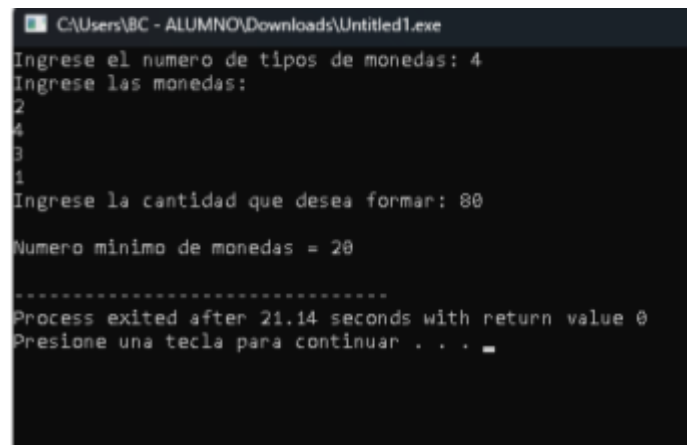
Aquí, $DP[x]$ significa cantidad mínima de monedas para alcanzar el valor de “X”. Como se observa, por cada valor (c_i) que se utiliza, se agrega una moneda a la contabilización; además, en el nuevo estado DP se le resta la cantidad c_i que se añadió.

La complejidad computacional se puede calcular a partir de la siguiente ecuación:

$$T(A, n) = \sum_{x=1}^X \sum_{i=1}^n c$$

Como en algunos casos anteriores, c es una constante y es el número de operaciones que se realizan dentro del algoritmo. Además, " n " representa la cantidad de tipos de monedas que se va a considerar en el problema. De esta manera, si cada operación vale $O(1)$, la complejidad total valdría $O(Xn)$.

Se muestra parte de una solución de este problema en el lenguaje C, plataforma Dev-C.



```
CAUsers\BC - ALUMNO\Downloads\Untitled1.exe
Ingrese el numero de tipos de monedas: 4
Ingrese las monedas:
2
4
3
1
Ingrese la cantidad que desea formar: 80
Numero minimo de monedas = 20
-----
Process exited after 21.14 seconds with return value 0
Presione una tecla para continuar . . .
```

Salida 8. Ejecución de Cambio de monedas

7. Bibliografía

- Bellman, R. (1957). *Dynamic Programming*. Princeton University Press.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4.^a ed.). MIT Press.
- Glosarix. (s. f.). *Subestructura óptima*. <https://glosarix.com/glossary/subestructura-optima/>
- Intellipaat. (2023). *Dynamic Programming: Characteristics, Approaches, and Examples*. Intellipaat Blog. <https://intellipaat.com/blog/dynamic-programming/>
- Interview Cake. (s. f.). *Overlapping Subproblems*. <https://www.interviewcake.com/concept/java/overlapping-subproblems>
- Lifeder. (2022). *Programación dinámica: concepto, características, ejemplos*. <https://www.lifeder.com/programacion-dinamica/>